```
"""
Ekmek Teknesi – Seviye Takip Botu (v1)

Her çalıştığında:
  1. Kripto (Binance) ve BIST (Yahoo) mum verisini çeker
  2. HTF tepe/dip, pivot, yatay alan, order block, FVG, likidite, BOS/CHoCH, Fibo
  3. Fiyat bir bölgeye YENİ girdiyse veya yeni bir yapı kırılımı/sweep olduysa Te
  4. Ne gönderdiğini state.json'a kaydeder (aynı bildirimi tekrar tekrar atmamak

Ayarlar için ayarlar.py dosyasına bak.
"""
import html
import json
import os
import sys
import time
from datetime import datetime, timedelta, timezone

import numpy as np
import pandas as pd
import requests

import ayarlar as A

TR = timezone(timedelta(hours=3))
DURUM_DOSYASI = "state.json"
OHLC = {"open": "first", "high": "max", "low": "min", "close": "last", "volume":
UA = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
                    "(KHTML, like Gecko) Chrome/124.0 Safari/537.36"}
TREND = {1: "🟢 Yükseliş", -1: "🔴 Düşüş", 0: "⚪ Belirsiz"}


# ================================================================
#  VERİ
# ================================================================
def _istek(url, params):
    for deneme in range(3):
        try:
            r = requests.get(url, params=params, headers=UA, timeout=20)
        except requests.RequestException:
            time.sleep(2 + 2 * deneme)
            continue
        if r.status_code == 200:
            return r.json()
```

```
        if r.status_code in (429, 500, 502, 503, 504):
            time.sleep(2 + 2 * deneme)
            continue
        return None
    return None


def binance_mum(sembol, aralik, limit):
    for taban in ("https://data-api.binance.vision", "https://api.binance.com"):
        d = _istek(f"{taban}/api/v3/klines", {"symbol": sembol, "interval": arali
        if d:
            df = pd.DataFrame([r[:6] for r in d], columns=["t", "open", "high", "
            df.index = pd.to_datetime(df.pop("t"), unit="ms", utc=True)
            return df.astype(float)
    return None


def yahoo_mum(sembol, aralik, donem):
    for host in ("query1", "query2"):
        d = _istek(f"https://{host}.finance.yahoo.com/v8/finance/chart/{sembol}",
                   {"interval": aralik, "range": donem})
        try:
            s = d["chart"]["result"][0]
            q = s["indicators"]["quote"][0]
            df = pd.DataFrame({k: q[k] for k in ("open", "high", "low", "close",
                               index=pd.to_datetime(s["timestamp"], unit="s", utc=
            df = df.astype(float).dropna(subset=["open", "high", "low", "close"])
            if len(df) > 20:
                return df
        except (TypeError, KeyError, IndexError, ValueError):
            pass
    return None


def veri_al(piyasa, sembol, ysembol):
    """(4 saatlik, günlük) mum tablolarını döndürür."""
    if piyasa == "kripto":
        h4 = binance_mum(sembol, "4h", 500)
        d1 = binance_mum(sembol, "1d", 1000)
        if h4 is not None and d1 is not None:
            return h4, d1
        h1, d1 = yahoo_mum(ysembol, "1h", "3mo"), yahoo_mum(ysembol, "1d", "5y")
        if h1 is None or d1 is None:
            return None
        return h1.resample("4h").agg(OHLC).dropna(), d1

    # BIST: 4S mumlar 10:00 / 14:00 TR'de başlar (UTC 07:00 / 11:00)
```

```
    h1, d1 = yahoo_mum(ysembol, "1h", "6mo"), yahoo_mum(ysembol, "1d", "5y")
    if h1 is None or d1 is None:
        return None
    h4 = h1.resample("4h", offset="7h").agg(OHLC).dropna()
    gun = d1.index.tz_convert("Europe/Istanbul").strftime("%Y-%m-%d")
    d1.index = pd.to_datetime(gun).tz_localize("UTC")
    d1 = d1[~d1.index.duplicated(keep="last")]
    return h4, d1


# ====================================================================
#  GÖSTERGELER
# ====================================================================
def atr(df, n=14):
    pc = df.close.shift()
    tr = pd.concat([df.high - df.low, (df.high - pc).abs(), (df.low - pc).abs()],
    return float(tr.rolling(n).mean().iloc[-1])


def salinimlar(df, n):
    """Swing tepe ve dip indeksleri (solunda ve sağında n mum daha düşük/yüksek).
    h, l = df.high.values, df.low.values
    sh, sl = [], []
    for i in range(n, len(df) - n):
        if h[i] == h[i - n:i + n + 1].max():
            sh.append(i)
        if l[i] == l[i - n:i + n + 1].min():
            sl.append(i)
    return sh, sl


def yapi(df, n):
    """Market structure: kapanışla kırılan swing'ler -> BOS / CHoCH olayları."""
    h, l, c = df.high.values, df.low.values, df.close.values
    sh, sl = salinimlar(df, n)
    sh, sl = set(sh), set(sl)
    trend, son_t, son_d, olaylar = 0, None, None, []
    for i in range(len(df)):
        j = i - n  # bu mumda onaylanan swing
        if j in sh:
            son_t = [j, h[j], False]
        if j in sl:
            son_d = [j, l[j], False]
        if son_t and not son_t[2] and c[i] > son_t[1]:
            olaylar.append({"i": i, "yon": 1, "tur": "CHoCH" if trend == -1 else
                            "seviye": son_t[1], "k": son_t[0]})
            son_t[2], trend = True, 1
```

```
        if son_d and not son_d[2] and c[i] < son_d[1]:
            olaylar.append({"i": i, "yon": -1, "tur": "CHoCH" if trend == 1 else
                            "seviye": son_d[1], "k": son_d[0]})
            son_d[2], trend = True, -1
    return trend, olaylar


def order_bloklar(df, olaylar, tf):
    """Kırılımı başlatan son ters mum = order block. Sadece henüz dokunulmamış ol
    o, h, l, c = df.open.values, df.high.values, df.low.values, df.close.values
    out = []
    for e in olaylar[-15:]:
        k, i = e["k"], e["i"]
        if e["yon"] == 1:
            m = k + int(np.argmin(l[k:i + 1]))
            t = next((x for x in range(m, max(k - 5, -1), -1) if c[x] < o[x]), m)
            lo, hi = l[t], h[t]
            if i + 1 < len(df) and l[i + 1:].min() <= hi:
                continue
        else:
            m = k + int(np.argmax(h[k:i + 1]))
            t = next((x for x in range(m, max(k - 5, -1), -1) if c[x] > o[x]), m)
            lo, hi = l[t], h[t]
            if i + 1 < len(df) and h[i + 1:].max() >= lo:
                continue
        ad = f"{tf} {'Bullish' if e['yon'] == 1 else 'Bearish'} Order Block"
        out.append({"ad": ad, "lo": lo, "hi": hi, "key": f"ob|{tf}|{df.index[t].i
    return out[-4:]


def fvgler(df, tf, a, son=150):
    """Doldurulmamış Fair Value Gap'ler (kısmen dolanlar küçülür, tamamen dolanla
    h, l = df.high.values, df.low.values
    out = []
    for i in range(max(2, len(df) - son), len(df)):
        if l[i] > h[i - 2] and l[i] - h[i - 2] >= 0.1 * a:
            lo, hi = h[i - 2], l[i]
            if i + 1 < len(df):
                mn = l[i + 1:].min()
                if mn <= lo:
                    continue
                hi = min(hi, mn)
            out.append({"ad": f"{tf} Bullish FVG", "lo": lo, "hi": hi,
                        "key": f"fvg|{tf}|{df.index[i].isoformat()}"})
        if h[i] < l[i - 2] and l[i - 2] - h[i] >= 0.1 * a:
            lo, hi = h[i], l[i - 2]
            if i + 1 < len(df):
```

```
                mx = h[i + 1:].max()
                if mx >= hi:
                    continue
                lo = max(lo, mx)
            out.append({"ad": f"{tf} Bearish FVG", "lo": lo, "hi": hi,
                        "key": f"fvg|{tf}|{df.index[i].isoformat()}"})
    return out[-6:]


def onceki_mum(df, kural, ref):
    x = df.resample(kural).agg(OHLC).dropna() if kural else df
    x = x[x.index < ref]
    return x.iloc[-1] if len(x) else None


def htf_ve_pivot(d1, ref):
    seviyeler, pivotlar = [], []
    donemler = [("Önceki Gün", None, "G"), ("Önceki Hafta", "W-SUN", "H"),
                ("Önceki Ay", "ME", "A"), ("Önceki Yıl", "YE", "Y")]
    for ad, kural, kisa in donemler:
        m = onceki_mum(d1, kural, ref)
        if m is None:
            continue
        seviyeler.append({"ad": f"{ad} Tepesi (P{kisa}H)", "lo": m.high, "hi": m.
        seviyeler.append({"ad": f"{ad} Dibi (P{kisa}L)", "lo": m.low, "hi": m.low
        if kisa in ("G", "H"):
            p = (m.high + m.low + m.close) / 3
            r = m.high - m.low
            isim = "Günlük" if kisa == "G" else "Haftalık"
            for etiket, deger in (("P", p), ("R1", 2 * p - m.low), ("S1", 2 * p -
                                  ("R2", p + r), ("S2", p - r)):
                pivotlar.append({"ad": f"{isim} Pivot {etiket}", "lo": deger, "hi
    return seviyeler, pivotlar


def yatay_alanlar(d1, a_gun):
    x = d1.iloc[-500:]
    sh, sl = salinimlar(x, A.SWING_GENISLIK)
    noktalar = sorted([x.high.values[i] for i in sh] + [x.low.values[i] for i in
    tol = 0.35 * a_gun
    kumeler = []
    for p in noktalar:
        if kumeler and p - kumeler[-1][0] <= tol:
            kumeler[-1].append(p)
        else:
            kumeler.append([p])
    out = []
```

```
    for k in kumeler:
        if len(k) >= A.MIN_DOKUNMA:
            orta = (min(k) + max(k)) / 2
            out.append({"ad": f"Yatay Alan ({len(k)} dokunuş)", "lo": min(k), "hi
                        "key": f"yatay|{round(orta / tol)}"})
    return out


def likidite(h4, a4, son=150):
    """Eşit tepe/dipler (EQH/EQL) ve son kapanan mumlardaki likidite süpürmeleri.
    bas = max(0, len(h4) - son)
    x = h4.iloc[bas:]
    h, l, c = x.high.values, x.low.values, x.close.values
    sh, sl = salinimlar(x, A.SWING_GENISLIK)
    tol = 0.15 * a4
    seviyeler, olaylar = [], []

    for idx, fiyat, yon in ((sh, h, 1), (sl, l, -1)):
        son_sw = idx[-10:]
        bulunan = []
        for a_i in range(len(son_sw)):
            for b_i in range(a_i + 1, len(son_sw)):
                i, j = son_sw[a_i], son_sw[b_i]
                if abs(fiyat[i] - fiyat[j]) > tol:
                    continue
                lvl = max(fiyat[i], fiyat[j]) if yon == 1 else min(fiyat[i], fiya
                bozuldu = fiyat[i + 1:].max() > lvl if yon == 1 else fiyat[i + 1:
                if not bozuldu and all(abs(lvl - b) > tol for b in bulunan):
                    bulunan.append(lvl)
        for lvl in bulunan:
            ad = "Eşit Tepeler (EQH) – üst likidite" if yon == 1 else "Eşit Diple
            seviyeler.append({"ad": ad, "lo": lvl, "hi": lvl})

    # Sweep: fitil seviyenin ötesine geçip kapanış geri içeride
    for t in (len(x) - 2, len(x) - 1):
        for s in sh:
            if s < t - 1 and (s + 1 >= t or h[s + 1:t].max() < h[s]) and h[t] > h
                olaylar.append({"metin": f"💧 4S üst likidite süpürüldü ({fmt(h[s
                                "key": f"sweep|4S|up|{x.index[t].isoformat()}"})
                break
        for s in sl:
            if s < t - 1 and (s + 1 >= t or l[s + 1:t].min() > l[s]) and l[t] < l
                olaylar.append({"metin": f"💧 4S alt likidite süpürüldü ({fmt(l[s
                                "key": f"sweep|4S|dn|{x.index[t].isoformat()}"})
                break
    return seviyeler, olaylar
```

```
def fibonacci(d1, a_gun):
    x = d1.iloc[-A.FIB_GUN:]
    ih, il = int(np.argmax(x.high.values)), int(np.argmin(x.low.values))
    H, L = x.high.values[ih], x.low.values[il]
    if H - L < 3 * a_gun:
        return []
    yukselis = il < ih
    dalga = "yükseliş" if yukselis else "düşüş"

    def seviye(r):
        return H - r * (H - L) if yukselis else L + r * (H - L)

    k = f"fib|{fmt(H)}|{fmt(L)}"
    gp = sorted([seviye(0.618), seviye(0.65)])
    return [
        {"ad": f"Fib 0.5 ({dalga} dalgası)", "lo": seviye(0.5), "hi": seviye(0.5)
        {"ad": f"Fib Golden Pocket 0.618–0.65 ({dalga} dalgası)", "lo": gp[0], "h
        {"ad": f"Fib 0.786 ({dalga} dalgası)", "lo": seviye(0.786), "hi": seviye(
    ]


# ═══════════════════════════════════════════════════════════════════
#  ANALİZ
# ═══════════════════════════════════════════════════════════════════
def kapanmis(df, sure, simdi):
    return df[df.index + sure <= simdi]


def analiz_et(h4, d1, piyasa):
    simdi = pd.Timestamp.now(tz="UTC")
    fiyat = float(h4.close.iloc[-1])
    if piyasa == "bist":
        bugun = pd.Timestamp(datetime.now(TR).strftime("%Y-%m-%d"), tz="UTC")
        tr_saat = datetime.now(TR)
        kapanis_gecti = (tr_saat.hour, tr_saat.minute) >= (18, 15)
        d1c = d1[(d1.index < bugun) | ((d1.index == bugun) & kapanis_gecti)]
        ref = bugun
    else:
        d1c = kapanmis(d1, pd.Timedelta("1D"), simdi)
        ref = simdi.normalize()
    h4c = kapanmis(h4, pd.Timedelta("4h"), simdi)

    a4, a_gun = atr(h4c), atr(d1c)
    n = A.SWING_GENISLIK
    trend4, olay4 = yapi(h4c, n)
    trendG, olayG = yapi(d1c, n)
```

```
    seviyeler, olaylar = [], []
    htf, piv = htf_ve_pivot(d1, ref)
    if A.HTF_SEVIYELER:
        seviyeler += htf
    if A.PIVOTLAR:
        seviyeler += piv
    if A.YATAY_ALANLAR:
        seviyeler += yatay_alanlar(d1c, a_gun)
    if A.ORDER_BLOCK:
        seviyeler += order_bloklar(h4c, olay4, "4S") + order_bloklar(d1c, olayG,
    if A.FVG:
        seviyeler += fvgler(h4c, "4S", a4) + fvgler(d1c, "Günlük", a_gun)
    if A.LIKIDITE:
        s, o = likidite(h4c, a4)
        seviyeler += s
        olaylar += o
    if A.FIBONACCI:
        seviyeler += fibonacci(d1c, a_gun)

    if A.YAPI:
        for df, ol, tf in ((h4c, olay4, "4S"), (d1c, olayG, "Günlük")):
            for e in ol:
                if e["i"] >= len(df) - 2:
                    ok = "⬆️" if e["yon"] == 1 else "⬇️"
                    anlam = ("trend dönüşü sinyali" if e["tur"] == "CHoCH"
                             else ("yükseliş yapısı devam" if e["yon"] == 1 else
                    olaylar.append({"metin": f"{ok} {tf} {e['tur']} – {fmt(e['sev
                                    "key": f"yapi|{tf}|{e['yon']}|{df.index[e['i'

    for s in seviyeler:
        s.setdefault("key", f"{s['ad']}|{s['lo']:.6g}")
    return {"fiyat": fiyat, "tol": A.TOLERANS_ATR * a4, "onceki": float(h4c.close
            "seviyeler": seviyeler, "olaylar": olaylar, "trend4": trend4, "trendG


# ═══════════════════════════════════════════════════════════════════
#  BİLDİRİM
# ═══════════════════════════════════════════════════════════════════
def fmt(x):
    if x >= 1000:
        s = f"{x:,.0f}"
    elif x >= 1:
        s = f"{x:,.2f}"
    else:
        s = f"{x:,.5f}"
    return s.replace(",", "X").replace(".", ",").replace("X", ".")
```

```
def yuzde(x, fiyat):
    return f"{(x / fiyat - 1) * 100:+.1f}%".replace(".", ",")


def aralik(s):
    return fmt(s["lo"]) if abs(s["hi"] - s["lo"]) < 1e-12 else f"{fmt(s['lo'])} –


def link(piyasa, sembol):
    borsa = "BINANCE" if piyasa == "kripto" else "BIST"
    return f'<a href="https://www.tradingview.com/chart/?symbol={borsa}:{sembol}"


def kontrol(sembol, an, durum, ilk):
    fiyat, tol = an["fiyat"], an["tol"]
    eski = durum["icerde"].get(sembol, {})
    yeni, satirlar = {}, []
    simdi = datetime.now(TR).isoformat(timespec="minutes")
    for s in an["seviyeler"]:
        k = s["key"]
        if s["lo"] - tol <= fiyat <= s["hi"] + tol:
            yeni[k] = eski.get(k, simdi)
            if k not in eski and not ilk:
                if an["onceki"] > s["hi"]:
                    yon = " · yukarıdan geldi (destek testi)"
                elif an["onceki"] < s["lo"]:
                    yon = " · aşağıdan geldi (direnç testi)"
                else:
                    yon = ""
                satirlar.append(f"📍 <b>{html.escape(s['ad'])}</b>: {aralik(s)}{y
        elif k in eski and s["lo"] - 2 * tol <= fiyat <= s["hi"] + 2 * tol:
            yeni[k] = eski[k]  # bölgenin hemen dışında: tekrar bildirim atmamak
    durum["icerde"][sembol] = yeni

    for o in an["olaylar"]:
        k = f"{sembol}|{o['key']}"
        if k not in durum["olaylar"]:
            durum["olaylar"][k] = simdi
            if not ilk:
                satirlar.append(html.escape(o["metin"]))
    return satirlar


def ozet_satiri(sembol, piyasa, an):
    fiyat = an["fiyat"]
```

```
    sv = an["seviyeler"]
    ust = sorted([s for s in sv if s["lo"] > fiyat], key=lambda s: s["lo"])[:3]
    alt = sorted([s for s in sv if s["hi"] < fiyat], key=lambda s: -s["hi"])[:3]
    ic = [s for s in sv if s["lo"] <= fiyat <= s["hi"]]
    m = [f"<b>{sembol}</b> · {fmt(fiyat)}",
         f"Trend: 4S {TREND[an['trend4']]} | Günlük {TREND[an['trendG']]}"]
    for s in reversed(ust):
        m.append(f"  🔺 {aralik(s)} ({yuzde(s['lo'], fiyat)}) {html.escape(s['ad'
    for s in ic:
        m.append(f"  ▶️ ŞU AN İÇİNDE: {aralik(s)} {html.escape(s['ad'])}")
    for s in alt:
        m.append(f"  🔻 {aralik(s)} ({yuzde(s['hi'], fiyat)}) {html.escape(s['ad'
    m.append(link(piyasa, sembol))
    return "\n".join(m)


def gonder(metin):
    token, chat = os.getenv("TELEGRAM_TOKEN"), os.getenv("TELEGRAM_CHAT_ID")
    if not token or not chat:
        print("---- (Telegram ayarlı değil, mesaj ekrana yazılıyor) ----\n" + met
        return True
    parcalar, parca = [], ""
    for satir in metin.split("\n"):
        if len(parca) + len(satir) > 3800:
            parcalar.append(parca)
            parca = ""
        parca += satir + "\n"
    parcalar.append(parca)
    basarili = True
    for p in parcalar:
        r = requests.post(f"https://api.telegram.org/bot{token}/sendMessage",
                          json={"chat_id": chat, "text": p, "parse_mode": "HTML",
                                "disable_web_page_preview": True}, timeout=20)
        if r.status_code != 200:
            print("Telegram hatası:", r.status_code, r.text[:300])
            basarili = False
    return basarili


# ==========================================================================
#  ANA AKIŞ
# ==========================================================================
def durum_yukle():
    try:
        with open(DURUM_DOSYASI, encoding="utf-8") as f:
            d = json.load(f)
    except (FileNotFoundError, json.JSONDecodeError):
```

```
        d = {}
    for k, v in (("icerde", {}), ("olaylar", {}), ("gorulen", []), ("son_ozet", "
        d.setdefault(k, v)
    sinir = (datetime.now(TR) - timedelta(days=10)).isoformat()
    d["olaylar"] = {k: v for k, v in d["olaylar"].items() if v >= sinir}
    return d


def main():
    durum = durum_yukle()
    simdi_tr = datetime.now(TR)
    bugun = simdi_tr.strftime("%Y-%m-%d")
    manuel = os.getenv("MANUEL") == "true"
    ozet_zamani = manuel or (A.GUNLUK_OZET_SAATI is not None
                             and simdi_tr.hour >= A.GUNLUK_OZET_SAATI
                             and durum["son_ozet"] != bugun)

    semboller = [("kripto", s, y) for s, y in A.KRIPTO.items()] + \
                [("bist", s, f"{s}.IS") for s in A.BIST]
    ozetler, hatalar, basarili = [], [], True

    for piyasa, sembol, ysembol in semboller:
        try:
            veri = veri_al(piyasa, sembol, ysembol)
            if veri is None:
                hatalar.append(sembol)
                continue
            an = analiz_et(*veri, piyasa)
            ilk = sembol not in durum["gorulen"]
            satirlar = kontrol(sembol, an, durum, ilk)
            if ilk:
                durum["gorulen"].append(sembol)
            if satirlar:
                mesaj = f"🔔 <b>{sembol}</b> · {fmt(an['fiyat'])}\n" + "\n".join(
                        "\n" + link(piyasa, sembol)
                basarili &= gonder(mesaj)
            if ozet_zamani:
                ozetler.append(ozet_satiri(sembol, piyasa, an))
        except Exception as e:  # bir sembol hata verirse diğerleri devam etsin
            print(f"{sembol} hata: {e!r}")
            hatalar.append(sembol)

    if ozet_zamani:
        baslik = f"📊 <b>Seviye Özeti</b> · {simdi_tr.strftime('%d.%m.%Y %H:%M')}
        metin = baslik + "\n\n" + "\n\n".join(ozetler)
        if hatalar:
            metin += f"\n\n⚠️ Veri alınamadı: {', '.join(hatalar)}"
```

```
            basarili &= gonder(metin)
            durum["son_ozet"] = bugun

        with open(DURUM_DOSYASI, "w", encoding="utf-8") as f:
            json.dump(durum, f, ensure_ascii=False, indent=1)

        if hatalar:
            print("Veri alınamayan semboller:", hatalar)
        if not basarili:
            sys.exit("Telegram'a mesaj gönderilemedi – TOKEN ve CHAT_ID ayarlarını ko


if __name__ == "__main__":
    main()
```